

Solución Completa: Bloqueo Automático de Puntos de Venta por Mora y Gestión de Cobranza

A continuación te presento una solución detallada para cada uno de los 6 puntos que planteas, con el código completo listo para implementar en tu entorno Oracle (PL/SQL Developer + Oracle Forms Builder sobre Windows). He diseñado los objetos de base de datos con datos ficticios que respetan la relación jerárquica: **CANAL** → **DIVISION** → **TERRITORIO** → **AGENCIA** → **PUNTO_VENTA** → **CLIENTE**, y las tablas transaccionales **FACTURA** y **SERVICIO**.

Modelo de Datos Ficticio

Antes de las soluciones, creamos las tablas con datos de ejemplo. Esto te permite probar cada proceso sin afectar datos reales.

```
sql

-- =====
-- 1. TABLAS MAESTRAS (Jerarquía)
-- =====

CREATE TABLE canal (
    id_canal      NUMBER PRIMARY KEY,
    nombre_canal  VARCHAR2(100) NOT NULL
);

CREATE TABLE division (
    id_division   NUMBER PRIMARY KEY,
    id_canal      NUMBER NOT NULL,
    nombre_division VARCHAR2(100) NOT NULL,
    CONSTRAINT fk_div_canal FOREIGN KEY (id_canal) REFERENCES canal(id_canal)
);

CREATE TABLE territorio (
    id_territorio NUMBER PRIMARY KEY,
    id_division   NUMBER NOT NULL,
    nombre_territorio VARCHAR2(100),
    CONSTRAINT fk_terr_div FOREIGN KEY (id_division) REFERENCES division(id_division)
);
```

```

CREATE TABLE agencia (
    id_agencia      NUMBER PRIMARY KEY,
    id_territorio   NUMBER NOT NULL,
    nombre_agencia  VARCHAR2(100),
    CONSTRAINT fk_agencia_terr FOREIGN KEY (id_territorio) REFERENCES
territorio(id_territorio)
);

CREATE TABLE punto_venta (
    id_punto_venta  NUMBER PRIMARY KEY,
    id_agencia      NUMBER NOT NULL,
    nombre_pv       VARCHAR2(150),
    estado          VARCHAR2(10) DEFAULT 'ACTIVO' NOT NULL,
    direccion       VARCHAR2(200),
    CONSTRAINT fk_pv_agencia FOREIGN KEY (id_agencia) REFERENCES agencia(id_agencia),
    CONSTRAINT ck_pv_estado CHECK (estado IN ('ACTIVO', 'INACTIVO'))
);

CREATE TABLE cliente (
    id_cliente      NUMBER PRIMARY KEY,
    id_punto_venta  NUMBER NOT NULL,
    identificacion  VARCHAR2(20),
    nombre_cliente  VARCHAR2(200),
    email          VARCHAR2(150),
    telefono        VARCHAR2(20),
    CONSTRAINT fk_cli_pv FOREIGN KEY (id_punto_venta) REFERENCES punto_venta(id_punto_venta)
);

-- =====
-- 2. TABLAS TRANSACCIONALES
-- =====

CREATE TABLE servicio (
    id_servicio     NUMBER PRIMARY KEY,
    id_cliente      NUMBER NOT NULL,
    id_punto_venta  NUMBER NOT NULL,  -- Campo clave para el filtro
    descripcion     VARCHAR2(200),
    monto_anual     NUMBER(12,2),
    fecha_compra    DATE,
    tipo_pago       VARCHAR2(20) DEFAULT 'CREDITO',
    CONSTRAINT fk_serv_cli FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente),

```

```

        CONSTRAINT fk_serv_pv FOREIGN KEY (id_punto_venta) REFERENCES
punto_venta(id_punto_venta)
);

```

```

CREATE TABLE factura (
    id_factura      NUMBER PRIMARY KEY,
    id_servicio     NUMBER NOT NULL,
    numero_factura  VARCHAR2(30),
    monto_total     NUMBER(12,2),
    saldo_pendiente NUMBER(12,2),
    fecha_emision   DATE,
    fecha_vencimiento DATE,
    estado          VARCHAR2(15) DEFAULT 'PENDIENTE',
    CONSTRAINT fk_fact_serv FOREIGN KEY (id_servicio) REFERENCES servicio(id_servicio),
    CONSTRAINT ck_fact_estado CHECK (estado IN ('PENDIENTE','PAGADA','VENCIDA','EN_MORA'))
);

```

```

-- =====
-- 3. TABLA DE ARREGLOS DE PAGO
-- =====

```

```

CREATE TABLE arreglo_pago (
    id_arreglo      NUMBER PRIMARY KEY,
    id_cliente       NUMBER NOT NULL,
    fecha_acuerdo    DATE DEFAULT SYSDATE,
    monto_acordado   NUMBER(12,2),
    num_cuotas       NUMBER(2),
    estado           VARCHAR2(15) DEFAULT 'ACTIVO',
    CONSTRAINT fk_arr_cli FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
);

```

```

-- =====
-- 4. TABLA DE HISTORIAL / AUDITORÍA
-- =====

```

```

CREATE TABLE historial_bloqueo_pv (
    id_historial     NUMBER PRIMARY KEY,
    id_punto_venta   NUMBER NOT NULL,
    estado_anterior  VARCHAR2(10),
    estado_nuevo     VARCHAR2(10),
    motivo           VARCHAR2(200),
    usuario_bd       VARCHAR2(50) DEFAULT USER,
    usuario_app      VARCHAR2(50),

```

```

    fecha_accion      TIMESTAMP DEFAULT SYSTIMESTAMP,
    origen            VARCHAR2(20),  -- 'AUTOMATICO' o 'MANUAL'
    id_cliente        NUMBER,
    dias_mora         NUMBER,
    observaciones     VARCHAR2(500),
    CONSTRAINT fk_hist_pv FOREIGN KEY (id_punto_venta) REFERENCES
punto_venta(id_punto_venta)
);

-- =====
-- 5. SECUENCIAS
-- =====

CREATE SEQUENCE seq_historial_bloqueo START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_arreglo_pago START WITH 1 INCREMENT BY 1;

```

Datos de Ejemplo (Demo)

```

sql

-- Canales
INSERT INTO canal VALUES (1, 'Canal Retail');
INSERT INTO canal VALUES (2, 'Canal Corporativo');

-- Divisiones (la clave: 'Clientes de Mi Negocio' pertenece al Canal 1)
INSERT INTO division VALUES (10, 1, 'Clientes de Mi Negocio');
INSERT INTO division VALUES (20, 1, 'Grandes Cuentas');
INSERT INTO division VALUES (30, 2, 'Corporativo Premium');

-- Territorios
INSERT INTO territorio VALUES (100, 10, 'Territorio Norte');
INSERT INTO territorio VALUES (101, 10, 'Territorio Sur');
INSERT INTO territorio VALUES (102, 20, 'Territorio Centro');

-- Agencias
INSERT INTO agencia VALUES (1000, 100, 'Agencia Norte 1');
INSERT INTO agencia VALUES (1001, 100, 'Agencia Norte 2');
INSERT INTO agencia VALUES (1002, 101, 'Agencia Sur 1');

-- Puntos de Venta (todos ACTIVOS inicialmente)
INSERT INTO punto_venta VALUES (10000, 1000, 'PV Norte Centro', 'ACTIVO', 'Av. Norte 123');
INSERT INTO punto_venta VALUES (10001, 1000, 'PV Norte Plaza', 'ACTIVO', 'Calle 45 #12-30');

```

```

INSERT INTO punto_venta VALUES (10002, 1001, 'PV Norte Express', 'ACTIVO', 'Carrera 7 #80-15');
INSERT INTO punto_venta VALUES (10003, 1002, 'PV Sur Mayorista', 'ACTIVO', 'Av. Sur 456');

-- Clientes
INSERT INTO cliente VALUES (1, 10000, '900123456', 'Distribuidora El Sol', 'elsol@mail.com', '3001112233');
INSERT INTO cliente VALUES (2, 10001, '900789012', 'Comercial Luna', 'luna@mail.com', '3004445566');
INSERT INTO cliente VALUES (3, 10002, '901234567', 'Tienda Estrella', 'estrella@mail.com', '3007778899');
INSERT INTO cliente VALUES (4, 10003, '902345678', 'Supermercado Sur', 'sur@mail.com', '3009990011');

-- Servicios (compra anual, pago mensual a crédito)
-- Cliente 1: 3 facturas vencidas (mora alta)
INSERT INTO servicio VALUES (100, 1, 10000, 'Servicio Anual Premium', 1200000, SYSDATE-90, 'CREDITO');
INSERT INTO factura VALUES (1000, 100, 'F-2024-001', 100000, 100000, SYSDATE-90, SYSDATE-60, 'VENCIDA');
INSERT INTO factura VALUES (1001, 100, 'F-2024-002', 100000, 100000, SYSDATE-60, SYSDATE-30, 'VENCIDA');
INSERT INTO factura VALUES (1002, 100, 'F-2024-003', 100000, 100000, SYSDATE-30, SYSDATE, 'VENCIDA');

-- Cliente 2: 1 factura vencida (mora Leve)
INSERT INTO servicio VALUES (101, 2, 10001, 'Servicio Anual Básico', 600000, SYSDATE-60, 'CREDITO');
INSERT INTO factura VALUES (1003, 101, 'F-2024-004', 50000, 50000, SYSDATE-60, SYSDATE-15, 'VENCIDA');

-- Cliente 3: AL día (sin mora)
INSERT INTO servicio VALUES (102, 3, 10002, 'Servicio Anual Estándar', 900000, SYSDATE-30, 'CREDITO');
INSERT INTO factura VALUES (1004, 102, 'F-2024-005', 75000, 0, SYSDATE-30, SYSDATE+30, 'PAGADA');

-- Cliente 4: AL día
INSERT INTO servicio VALUES (103, 4, 10003, 'Servicio Anual Premium', 1200000, SYSDATE-15, 'CREDITO');

```

```
INSERT INTO factura VALUES (1005, 103, 'F-2024-006', 100000, 0, SYSDATE-15, SYSDATE+45, 'PAGADA');
```

```
COMMIT;
```

1. Bloqueo Automático de Puntos de Venta por Mora

Recomendación Técnica

La mejor opción es un **procedimiento almacenado** ejecutado por un **DBMS_SCHEDULER Job**. No recomiendo triggers porque:

- Un trigger se dispara por eventos DML, y aquí necesitas evaluar periódicamente la mora (no hay un evento puntual que lo dispare).
- Un job programado es determinista, auditable y no afecta el rendimiento de las transacciones diarias.
- El procedimiento puede ser invocado tanto por el job automático como por el formulario manual (punto 5), reutilizando la lógica.

Lógica del Proceso

1. Identificar todos los puntos de venta que pertenecen a la división '**Cientes de Mi Negocio**' (a través de la cadena jerárquica completa).
2. Para cada punto de venta, verificar si tiene clientes con facturas **VENCIDAS** y calcular los días de mora.
3. Si hay al menos 1 factura vencida, cambiar el estado del PV a '**INACTIVO**'.
4. Registrar la acción en la tabla de historial.

Código del Procedimiento

```
sql  
  
CREATE OR REPLACE PROCEDURE pr_bloqueo_automatico_pv (  
    p_dias_min_mora IN NUMBER DEFAULT 1,  
    p_usuario_app   IN VARCHAR2 DEFAULT 'SISTEMA'  
) IS
```

```
v_total_bloqueados NUMBER := 0;
```

```
BEGIN
```

```
-- Recorremos los PV de la división 'Clientes de Mi Negocio' que estén ACTIVOS
```

```
-- y que tengan clientes con facturas VENCIDAS
```

```
FOR r_pv IN (
```

```
    SELECT DISTINCT
```

```
        pv.id_punto_venta,
```

```
        pv.estado AS estado_actual,
```

```
        c.id_cliente,
```

```
        MIN(TRUNC(SYSDATE - f.fecha_vencimiento)) AS dias_mora_max
```

```
FROM punto_venta pv
```

```
JOIN agencia a          ON a.id_agencia = pv.id_agencia
```

```
JOIN territorio t        ON t.id_territorio = a.id_territorio
```

```
JOIN division d          ON d.id_division = t.id_division
```

```
JOIN cliente c           ON c.id_punto_venta = pv.id_punto_venta
```

```
JOIN servicio s          ON s.id_cliente = c.id_cliente
```

```
JOIN factura f           ON f.id_servicio = s.id_servicio
```

```
WHERE d.nombre_division = 'Clientes de Mi Negocio'
```

```
    AND pv.estado = 'ACTIVO'
```

```
    AND f.estado IN ('VENCIDA', 'EN_MORA')
```

```
    AND f.saldo_pendiente > 0
```

```
    AND f.fecha_vencimiento < TRUNC(SYSDATE)
```

```
    AND TRUNC(SYSDATE - f.fecha_vencimiento) >= p_dias_min_mora
```

```
GROUP BY pv.id_punto_venta, pv.estado, c.id_cliente
```

```
) LOOP
```

```
-- Actualizar el estado del punto de venta
```

```
UPDATE punto_venta
```

```
    SET estado = 'INACTIVO'
```

```
WHERE id_punto_venta = r_pv.id_punto_venta
```

```
    AND estado = 'ACTIVO';
```

```
-- Registrar en historial
```

```
INSERT INTO historial_bloqueo_pv (
```

```
    id_historial, id_punto_venta, estado_anterior, estado_nuevo,
```

```
    motivo, usuario_app, origen, id_cliente, dias_mora
```

```
) VALUES (
```

```
    seq_historial_bloqueo.NEXTVAL,
```

```
    r_pv.id_punto_venta,
```

```
    'ACTIVO',
```

```
    'INACTIVO',
```

```
    'Bloqueo automático por mora en facturas vencidas',
```

```

        p_usuario_app,
        'AUTOMATICO',
        r_pv.id_cliente,
        r_pv.dias_mora_max
    );

    v_total_bloqueados := v_total_bloqueados + 1;
END LOOP;

COMMIT;

DBMS_OUTPUT.PUT_LINE('Puntos de venta bloqueados: ' || v_total_bloqueados);
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20001, 'Error en bloqueo automático: ' || SQLERRM);
END pr_bloqueo_automatico_pv;
/

```

Job Programado para el Bloqueo Automático

```

sql

BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name          => 'JOB_BLOQUEO_AUTOMATICO_PV',
        job_type           => 'STORED_PROCEDURE',
        job_action         => 'PR_BLOQUEO_AUTOMATICO_PV',
        start_date         => SYSTIMESTAMP,
        repeat_interval    => 'FREQ=DAILY; BYHOUR=5; BYMINUTE=30; BYSECOND=0',
        enabled            => TRUE,
        comments           => 'Bloqueo diario de PV por mora - 5:30 AM'
    );
END;
/

```

Tiempo estimado: 4–6 horas (incluye pruebas con datos demo y validación de la lógica jerárquica).

2. Job Programado para Enviar Correo de Alerta por Mora

Tecnología Recomendada

Para enviar correos con contenido HTML y adjuntos desde Oracle en Windows, la opción más robusta es:

- **APEX_MAIL** si tienes Oracle APEX instalado (viene incluido en la mayoría de instalaciones de Oracle 11g en adelante). Es la más sencilla y soporta adjuntos BLOB/CLOB.
- **UTL_MAIL** si no tienes APEX. Es más limitado (32 KB para adjuntos en VARCHAR2), pero suficiente para reportes CSV pequeños.
- **UTL_SMTP** como última opción: control total, pero hay que construir el MIME manualmente.

Para tu caso recomiendo **APEX_MAIL** porque necesitas adjuntar un archivo Excel/CSV y el cuerpo HTML puede ser más rico.

Código del Procedimiento de Envío

sql

```
CREATE OR REPLACE PROCEDURE pr_enviar_alerta_mora
IS
    v_cuerpo_html    CLOB;
    v_asunto         VARCHAR2(200);
    v_mail_id        NUMBER;
    v_adjunto         BLOB;
    v_filename       VARCHAR2(100);
BEGIN
    -- Construir el cuerpo HTML con la Lista de PV en mora
    v_cuerpo_html := '<html><body>';
    v_cuerpo_html := v_cuerpo_html || '<h2>Reporte de Puntos de Venta en Mora</h2>';
    v_cuerpo_html := v_cuerpo_html || '<table border="1" cellpadding="5">';
    v_cuerpo_html := v_cuerpo_html || '<tr><th>Punto de Venta</th><th>Cliente</th>';
    v_cuerpo_html := v_cuerpo_html || '<th>Facturas Vencidas</th><th>Días Mora</th>';
    v_cuerpo_html := v_cuerpo_html || '<th>Saldo Total Mora</th></tr>';

    FOR r IN (
```

```

SELECT pv.nombre_pv,
       c.nombre_cliente,
       COUNT(f.id_factura) AS num_facturas,
       MAX(TRUNC(SYSDATE - f.fecha_vencimiento)) AS dias_mora,
       SUM(f.saldo_pendiente) AS saldo_mora
FROM punto_venta pv
JOIN agencia a      ON a.id_agencia = pv.id_agencia
JOIN territorio t    ON t.id_territorio = a.id_territorio
JOIN division d      ON d.id_division = t.id_division
JOIN cliente c       ON c.id_punto_venta = pv.id_punto_venta
JOIN servicio s      ON s.id_cliente = c.id_cliente
JOIN factura f       ON f.id_servicio = s.id_servicio
WHERE d.nombre_division = 'Clientes de Mi Negocio'
      AND pv.estado = 'INACTIVO'
      AND f.estado IN ('VENCIDA', 'EN_MORA')
      AND f.saldo_pendiente > 0
GROUP BY pv.nombre_pv, c.nombre_cliente
ORDER BY dias_mora DESC
) LOOP
    v_cuerpo_html := v_cuerpo_html || '<tr>';
    v_cuerpo_html := v_cuerpo_html || '<td>' || r.nombre_pv || '</td>';
    v_cuerpo_html := v_cuerpo_html || '<td>' || r.nombre_cliente || '</td>';
    v_cuerpo_html := v_cuerpo_html || '<td align="center">' || r.num_facturas ||
'</td>';
    v_cuerpo_html := v_cuerpo_html || '<td align="center">' || r.dias_mora || '</td>';
    v_cuerpo_html := v_cuerpo_html || '<td align="right">' || TO_CHAR(r.saldo_mora,
'L999G999G999D99') || '</td>';
    v_cuerpo_html := v_cuerpo_html || '</tr>';
END LOOP;

v_cuerpo_html := v_cuerpo_html || '</table></body></html>';

v_asunto := 'ALERTA: Puntos de Venta en Mora - ' || TO_CHAR(SYSDATE, 'DD/MM/YYYY');

-- Enviar el correo con APEX_MAIL
v_mail_id := APEX_MAIL.SEND(
    p_to      => 'cobranza1@empresa.com,cobranza2@empresa.com,gerencia@empresa.com',
    p_from    => 'alertas@empresa.com',
    p_subj    => v_asunto,
    p_body    => 'Reporte de puntos de venta en mora. Ver HTML.',
    p_body_html => v_cuerpo_html

```

```

);

-- Aquí se generaría y adjuntaría el CSV (ver punto 6 para el detalle)
-- v_adjunto := fn_generar_csv_mora;
-- v_filename := 'mora_' || TO_CHAR(SYSDATE, 'YYYYMMDD') || '.csv';
-- APEX_MAIL.ADD_ATTACHMENT(v_mail_id, v_adjunto, v_filename, 'text/csv');

APEX_MAIL.PUSH_QUEUE;

DBMS_OUTPUT.PUT_LINE('Correo enviado. Mail ID: ' || v_mail_id);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error enviando correo: ' || SQLERRM);
        RAISE;
END pr_enviar_alerta_mora;
/

```

Job Programado para Envío de Correo

```

sql

BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name          => 'JOB_ALERTA_MORA_CORREO',
        job_type           => 'STORED_PROCEDURE',
        job_action         => 'PR_ENVIAR_ALERTA_MORA',
        start_date         => SYSTIMESTAMP,
        repeat_interval    => 'FREQ=DAILY; BYHOUR=7; BYMINUTE=0; BYSECOND=0',
        enabled            => TRUE,
        comments           => 'Envío diario de alerta de mora - 7:00 AM'
    );
END;
/

```

Nota sobre configuración SMTP: Antes de usar `APEX_MAIL`, debes configurar el servidor SMTP en tu instancia Oracle. Ejecuta como SYS:

```

sql

BEGIN
    APEX_INSTANCE_ADMIN.SET_PARAMETER('SMTP_HOST_ADDRESS', 'smtp.tuempresa.com');
    APEX_INSTANCE_ADMIN.SET_PARAMETER('SMTP_HOST_PORT', '25');

```

```
APEX_INSTANCE_ADMIN.SET_PARAMETER('SMTP_USERNAME', 'alertas@empresa.com');
APEX_INSTANCE_ADMIN.SET_PARAMETER('SMTP_PASSWORD', 'tu_password');
COMMIT;
END;
/
```

Si no tienes APEX, usa UTL_MAIL:

```
sql

-- Habilitar UTL_MAIL (como SYS)
ALTER SYSTEM SET smtp_out_server = 'smtp.tuempresa.com:25' SCOPE=BOTH;
```

Tiempo estimado: 6–8 horas (incluye configuración SMTP, pruebas de envío y ajuste del HTML).

3. Desbloqueo Automático por Arreglo de Pago

Lógica

Cuando un cliente con mora accede a un **arreglo de pago**, su punto de venta debe reactivarse automáticamente. El procedimiento:

1. Detecta clientes que tienen un arreglo de pago **ACTIVO** creado en las últimas horas (o que acaban de registrarlo).
2. Verifica que el cliente tenga facturas vencidas.
3. Cambia el estado del PV asociado a '**ACTIVO**'.
4. Registra en el historial.

Código del Procedimiento

```
sql

CREATE OR REPLACE PROCEDURE pr_desbloqueo_por_arreglo (
    p_usuario_app IN VARCHAR2 DEFAULT 'SISTEMA'
) IS
    v_total_desbloqueados NUMBER := 0;
BEGIN
```

```

FOR r_pv IN (
    SELECT DISTINCT
        pv.id_punto_venta,
        c.id_cliente,
        ap.id_arreglo,
        ap.fecha_acuerdo
    FROM punto_venta pv
    JOIN agencia a      ON a.id_agencia = pv.id_agencia
    JOIN territorio t    ON t.id_territorio = a.id_territorio
    JOIN division d      ON d.id_division = t.id_division
    JOIN cliente c       ON c.id_punto_venta = pv.id_punto_venta
    JOIN arreglo_pago ap ON ap.id_cliente = c.id_cliente
    WHERE d.nombre_division = 'Clientes de Mi Negocio'
        AND pv.estado = 'INACTIVO'
        AND ap.estado = 'ACTIVO'
        AND ap.fecha_acuerdo >= TRUNC(SYSDATE) - 1 -- Arreglos recientes
) LOOP
    UPDATE punto_venta
        SET estado = 'ACTIVO'
    WHERE id_punto_venta = r_pv.id_punto_venta
        AND estado = 'INACTIVO';

    INSERT INTO historial_bloqueo_pv (
        id_historial, id_punto_venta, estado_anterior, estado_nuevo,
        motivo, usuario_app, origen, id_cliente
    ) VALUES (
        seq_historial_bloqueo.NEXTVAL,
        r_pv.id_punto_venta,
        'INACTIVO',
        'ACTIVO',
        'Desbloqueo automático por arreglo de pago ID: ' || r_pv.id_arreglo,
        p_usuario_app,
        'AUTOMATICO',
        r_pv.id_cliente
    );

    v_total_desbloqueados := v_total_desbloqueados + 1;
END LOOP;

COMMIT;

DBMS_OUTPUT.PUT_LINE('PV desbloqueados: ' || v_total_desbloqueados);

```

```

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20002, 'Error en desbloqueo: ' || SQLERRM);
END pr_desbloqueo_por_arreglo;
/

```

Job Programado (Cada 2 horas durante horario laboral)

```

sql

BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name          => 'JOB_DESBLOQUEO_ARREGLO',
        job_type           => 'STORED_PROCEDURE',
        job_action          => 'PR_DESBLOQUEO_POR_ARREGLO',
        start_date          => SYSTIMESTAMP,
        repeat_interval     => 'FREQ=HOURLY; INTERVAL=2; BYHOUR=8,10,12,14,16,18; BYMINUTE=0;
BYSECOND=0',
        enabled             => TRUE,
        comments            => 'Desbloqueo automático por arreglos de pago'
    );
END;
/

```

Tiempo estimado: 3–4 horas.

4. Tabla de Historial de Acciones

La tabla `historial_bloqueo_pv` ya fue creada en la sección del modelo de datos. Incluye todos los campos necesarios:

Campo	Descripción
<code>id_historial</code>	PK autogenerada
<code>id_punto_venta</code>	PV afectado
<code>estado_anterior</code>	Estado antes del cambio

Campo	Descripción
estado_nuevo	Estado después del cambio
motivo	Razón del cambio
usuario_bd	Usuario de base de datos (USER)
usuario_app	Usuario de la aplicación (Forms)
fecha_accion	Timestamp del cambio
origen	'AUTOMATICO' o 'MANUAL'
id_cliente	Cliente relacionado (si aplica)
dias_mora	Días de mora al momento del bloqueo
observaciones	Campo libre para notas

Trigger de Auditoría Adicional

Si deseas capturar **cualquier** cambio de estado en `punto_venta` (incluso los que no pasen por los procedimientos), puedes agregar un trigger de auditoría:

```
sql
```

```
CREATE OR REPLACE TRIGGER trg_audit_estado_pv
AFTER UPDATE OF estado ON punto_venta
FOR EACH ROW
WHEN (OLD.estado != NEW.estado)
DECLARE
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO historial_bloqueo_pv (
        id_historial, id_punto_venta, estado_anterior, estado_nuevo,
        motivo, usuario_bd, usuario_app, origen, observaciones
    ) VALUES (
        seq_historial_bloqueo.NEXTVAL,
        :NEW.id_punto_venta,
        :OLD.estado,
        :NEW.estado,
        'Cambio de estado detectado por trigger de auditoría',
```

```

    USER,
    SYS_CONTEXT('USERENV', 'CLIENT_IDENTIFIER'),
    'AUTOMATICO',
    'Trigger de auditoría - ' || TO_CHAR(SYSTIMESTAMP, 'DD/MM/YYYY HH24:MI:SS')
);
COMMIT;
END;
/

```

Precaución: El trigger con `PRAGMA AUTONOMOUS_TRANSACTION` evita que un rollback del procedimiento principal elimine el registro de auditoría. Sin embargo, si el procedimiento ya inserta en el historial, tendrías **duplicados**. Te recomiendo **usar solo una de las dos vías**: el procedimiento (que ya registra) o el trigger. Si usas el trigger, elimina los INSERT de historial de los procedimientos.

Tiempo estimado: 2–3 horas (incluye pruebas de auditoría).

5. Formulario Oracle Forms para Bloqueo/Desbloqueo Manual

Estructura del Formulario

Nombre del módulo: `FOR_BLOQUEO_PV.fmb`

Menú: `MENU_PRINCIPAL` → Opción `Cobranza` → Subopción `Gestión de Bloqueo de PV`

Bloques del Formulario

Bloque	Tipo	Basado en	Descripción
<code>BLK_PUNTOS_VENTA</code>	Data Block	<code>punto_venta</code> (query only)	Lista de PV de la división
<code>BLK_ACCION</code>	Control Block	N/A	Botones de acción y campo de observaciones

Código de los Triggers

Trigger WHEN-BUTTON-PRESSED del botón BTN_BLOQUEAR:

sql

DECLARE

```
v_id_pv      NUMBER := :BLK_PUNTOS_VENTA.id_punto_venta;  
v_estado     VARCHAR2(10) := :BLK_PUNTOS_VENTA.estado;  
v_usuario    VARCHAR2(50) := :GLOBAL.usuario_app; -- Variable global con el usuario  
v_obs        VARCHAR2(500) := :BLK_ACCION.observaciones;
```

BEGIN

```
IF v_estado = 'INACTIVO' THEN  
    MESSAGE('El punto de venta ya está bloqueado.');
```

RAISE FORM_TRIGGER_FAILURE;

```
END IF;
```

-- Actualizar estado

```
UPDATE punto_venta  
    SET estado = 'INACTIVO'  
    WHERE id_punto_venta = v_id_pv;
```

-- Registrar en historial

```
INSERT INTO historial_bloqueo_pv (  
    id_historial, id_punto_venta, estado_anterior, estado_nuevo,  
    motivo, usuario_bd, usuario_app, origen, observaciones  
) VALUES (  
    seq_historial_bloqueo.NEXTVAL,  
    v_id_pv,  
    'ACTIVO',  
    'INACTIVO',  
    'Bloqueo manual desde formulario',  
    USER,  
    v_usuario,  
    'MANUAL',  
    v_obs  
);
```

COMMIT;

```
MESSAGE('Punto de venta bloqueado correctamente.');
```

-- Refrescar el bloque

```

        :BLK_PUNTOS_VENTA.estado := 'INACTIVO';
    SYNCHRONIZE;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        MESSAGE('Error al bloquear: ' || SQLERRM);
        RAISE FORM_TRIGGER_FAILURE;
END;

```

Trigger WHEN-BUTTON-PRESSED del botón BTN_DESBLOQUEAR:

```

sql

DECLARE
    v_id_pv      NUMBER := :BLK_PUNTOS_VENTA.id_punto_venta;
    v_estado     VARCHAR2(10) := :BLK_PUNTOS_VENTA.estado;
    v_usuario    VARCHAR2(50) := :GLOBAL.usuario_app;
    v_obs        VARCHAR2(500) := :BLK_ACCION.observaciones;
BEGIN
    IF v_estado = 'ACTIVO' THEN
        MESSAGE('El punto de venta ya está activo.');
```

RAISE FORM_TRIGGER_FAILURE;

```

    END IF;

    UPDATE punto_venta
        SET estado = 'ACTIVO'
        WHERE id_punto_venta = v_id_pv;

    INSERT INTO historial_bloqueo_pv (
        id_historial, id_punto_venta, estado_anterior, estado_nuevo,
        motivo, usuario_bd, usuario_app, origen, observaciones
    ) VALUES (
        seq_historial_bloqueo.NEXTVAL,
        v_id_pv,
        'INACTIVO',
        'ACTIVO',
        'Desbloqueo manual desde formulario',
        USER,
        v_usuario,
        'MANUAL',
        v_obs
    );

```

```

        COMMIT;
        MESSAGE('Punto de venta desbloqueado correctamente.');
```

:BLK_PUNTOS_VENTA.estado := 'ACTIVO';

```

        SYNCHRONIZE;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        MESSAGE('Error al desbloquear: ' || SQLERRM);
        RAISE FORM_TRIGGER_FAILURE;
END;
```

Trigger WHEN-NEW-FORM-INSTANCE del formulario:

```

sql

BEGIN
    -- Establecer usuario de la aplicación (puedes obtenerlo del login)
    :GLOBAL.usuario_app := 'USUARIO_FORMS';
    -- Ejecutar query inicial
    EXECUTE_QUERY;
END;
```

Consulta Base del Bloque BLK_PUNTOS_VENTA

```

sql

SELECT pv.id_punto_venta,
       pv.nombre_pv,
       pv.estado,
       pv.direccion,
       a.nombre_agencia,
       t.nombre_territorio,
       d.nombre_division
FROM punto_venta pv
JOIN agencia a      ON a.id_agencia = pv.id_agencia
JOIN territorio t    ON t.id_territorio = a.id_territorio
JOIN division d      ON d.id_division = t.id_division
WHERE d.nombre_division = 'Clientes de Mi Negocio'
      AND (:BLK_ACCION.filtro_estado IS NULL
           OR pv.estado = :BLK_ACCION.filtro_estado)
ORDER BY pv.nombre_pv
```

Tiempo estimado: 8–10 horas (diseño del layout, bloques, triggers, pruebas de navegación y validación).

6. Proceso Automático Cada 3 Horas con Excel Adjunto

Recomendación de Tecnología

Componente	Tecnología Recomendada	Razón
Scheduler	DBMS_SCHEDULER.CREATE_JOB	Nativo de Oracle, confiable, auditable
Envío de correo	APEX_MAIL (o UTL_MAIL)	Soporta HTML y adjuntos BLOB
Generación de Excel	APEX_MAIL.ADD_ATTACHMENT con BLOB generado desde CLOB CSV	Más simple que crear XLSX real; compatible con Excel
Adjunto	CSV con extensión .csv (abre en Excel) o generar XLSX con AS_XLSX (PL/SQL nativo)	CSV es la opción más ligera y no requiere Java
Nota importante: DBMS_SCHEDULER.ADD_JOB_EMAIL_NOTIFICATION NO sirve para adjuntar archivos . Solo envía notificaciones de texto sobre el estado del job (inicio, fin, error). Para adjuntar un Excel/CSV, debes usar APEX_MAIL o UTL_MAIL dentro del procedimiento que ejecuta el job.		

Procedimiento Completo

sql

```
CREATE OR REPLACE PROCEDURE pr_reporte_mora_3h
IS
    v_mail_id      NUMBER;
    v_csv_clob     CLOB;
    v_csv_blob     BLOB;
    v_filename     VARCHAR2(100);
    v_cuerpo_html  CLOB;
    v_total_pv     NUMBER := 0;
    v_total_mora   NUMBER := 0;
```

```

BEGIN

-- =====
-- 1. CONSTRUIR EL CSV
-- =====

DBMS_LOB.CREATETEMPORARY(v_csv_clob, TRUE);

-- Cabecera del CSV

DBMS_LOB.WRITEAPPEND(v_csv_clob, LENGTH('Punto de Venta,Cliente,Facturas Vencidas,Dias
Mora,Saldo Total Mora' || CHR(10)),
                    'Punto de Venta,Cliente,Facturas Vencidas,Dias Mora,Saldo Total
Mora' || CHR(10));

FOR r IN (
    SELECT pv.nombre_pv,
           c.nombre_cliente,
           COUNT(f.id_factura) AS num_facturas,
           MAX(TRUNC(SYSDATE - f.fecha_vencimiento)) AS dias_mora,
           SUM(f.saldo_pendiente) AS saldo_mora
    FROM punto_venta pv
    JOIN agencia a      ON a.id_agencia = pv.id_agencia
    JOIN territorio t    ON t.id_territorio = a.id_territorio
    JOIN division d      ON d.id_division = t.id_division
    JOIN cliente c       ON c.id_punto_venta = pv.id_punto_venta
    JOIN servicio s      ON s.id_cliente = c.id_cliente
    JOIN factura f       ON f.id_servicio = s.id_servicio
    WHERE d.nombre_division = 'Clientes de Mi Negocio'
          AND pv.estado = 'INACTIVO'
          AND f.estado IN ('VENCIDA', 'EN_MORA')
          AND f.saldo_pendiente > 0
    GROUP BY pv.nombre_pv, c.nombre_cliente
    ORDER BY dias_mora DESC
) LOOP
    DECLARE
        v_linea VARCHAR2(500);
    BEGIN
        v_linea := ''' || r.nombre_pv || ',' ||
                   ''' || r.nombre_cliente || ',' ||
                   r.num_facturas || ',' ||
                   r.dias_mora || ',' ||
                   TO_CHAR(r.saldo_mora, '999999999.99') || CHR(10);
        DBMS_LOB.WRITEAPPEND(v_csv_clob, LENGTH(v_linea), v_linea);
    END
END

```

```

        v_total_pv      := v_total_pv + 1;
        v_total_mora    := v_total_mora + r.saldo_mora;

    END;
END LOOP;

-- Convertir CLOB a BLOB (UTF-8)
v_csv_blob := TO_BLOB(v_csv_clob);
v_filename := 'mora_pv_' || TO_CHAR(SYSDATE, 'YYYYMMDD_HH24MI') || '.csv';

-- =====
-- 2. CONSTRUIR CUERPO HTML DEL CORREO
-- =====

v_cuerpo_html := '<html><body>';
v_cuerpo_html := v_cuerpo_html || '<h2>Reporte de Mora - ' || TO_CHAR(SYSDATE,
'DD/MM/YYYY HH24:MI') || '</h2>';
v_cuerpo_html := v_cuerpo_html || '<p><b>Puntos de Venta en Mora:</b> ' || v_total_pv ||
'</p>';
v_cuerpo_html := v_cuerpo_html || '<p><b>Saldo Total en Mora:</b> ' ||
TO_CHAR(v_total_mora, 'L999G999G999D99') || '</p>';
v_cuerpo_html := v_cuerpo_html || '<p>Se adjunta el detalle en archivo CSV (abrir con
Excel).</p>';
v_cuerpo_html := v_cuerpo_html || '</body></html>';

-- =====
-- 3. ENVIAR CORREO CON ADJUNTO
-- =====

v_mail_id := APEX_MAIL.SEND(
    p_to      => 'cobranza1@empresa.com,cobranza2@empresa.com,gerencia@empresa.com',
    p_from    => 'alertas@empresa.com',
    p_subj    => 'Reporte Mora PV - ' || TO_CHAR(SYSDATE, 'DD/MM/YYYY HH24:MI'),
    p_body    => 'Reporte de mora. Ver HTML.',
    p_body_html => v_cuerpo_html
);

APEX_MAIL.ADD_ATTACHMENT(
    p_mail_id    => v_mail_id,
    p_attachment => v_csv_blob,
    p_filename   => v_filename,
    p_mime_type  => 'text/csv'
);

```

```

APEX_MAIL.PUSH_QUEUE;

DBMS_LOB.FREETEMPORARY(v_csv_clob);
DBMS_OUTPUT.PUT_LINE('Reporte enviado. PV: ' || v_total_pv || ', Mora: ' ||
v_total_mora);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
        RAISE;
END pr_reporte_mora_3h;
/

```

Job Cada 3 Horas desde las 6 AM

```

sql

BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name          => 'JOB_REPORTER_MORA_3H',
        job_type           => 'STORED_PROCEDURE',
        job_action         => 'PR_REPORTER_MORA_3H',
        start_date         => TRUNC(SYSDATE) + 6/24, -- Hoy a Las 6:00 AM
        repeat_interval    => 'FREQ=HOURLY; INTERVAL=3; BYHOUR=6,9,12,15,18,21; BYMINUTE=0;
BYSECOND=0',
        enabled            => TRUE,
        comments           => 'Reporte de mora cada 3 horas desde las 6 AM'
    );
END;
/

```

Nota sobre repeat_interval: Con `FREQ=HOURLY; INTERVAL=3; BYHOUR=6,9,12,15,18,21` el job se ejecutará exactamente a las 6:00, 9:00, 12:00, 15:00, 18:00 y 21:00. La última ejecución del día es a las 21:00, y la siguiente es a las 6:00 del día siguiente.

Tiempo estimado: 8–10 horas (incluye generación de CSV, conversión BLOB, pruebas de adjunto, validación de apertura en Excel).

Resumen de Tiempos Estimados

#	Tarea	Tiempo Estimado
1	Bloqueo automático por mora (procedimiento + job)	4–6 horas
2	Job de correo de alerta por mora	6–8 horas
3	Desbloqueo automático por arreglo de pago	3–4 horas
4	Tabla de historial + trigger de auditoría	2–3 horas
5	Formulario Oracle Forms (bloqueo/desbloqueo manual)	8–10 horas
6	Proceso cada 3 horas con Excel adjunto	8–10 horas
Total estimado		31–41 horas

Preguntas para Completar la Implementación

1. **¿Tienes Oracle APEX instalado?** Si no, debemos usar `UTL_MAIL` o `UTL_SMTP` (con las limitaciones de tamaño de adjunto).

2. **¿Cuál es el servidor SMTP y las credenciales de correo?** Necesito host, puerto, usuario y contraseña para configurar el envío.

3. **¿Cuál es el nombre exacto de la división?** Usé "Clientes de Mi Negocio" como ejemplo, pero si tiene otro nombre (con mayúsculas, tildes, etc.) hay que ajustar los WHERE.

4. **¿Los arreglos de pago se registran en una tabla existente?** Creé `arreglo_pago` como ejemplo. Si ya tienes una tabla con otra estructura, adapto el procedimiento.

5. **¿Quieres que el bloqueo sea por punto de venta completo o por cliente?** Actualmente bloqueo el PV completo. Si hay varios clientes en un PV y solo uno está en mora, ¿se debe bloquear todo el PV?

6. **¿El formulario de Oracle Forms es para Web (Oracle Forms 12c) o cliente-servidor (6i)?** Esto afecta la configuración de conexión y las variables globales.

7. **¿Tienes un directorio Oracle (DIRECTORY) configurado para escribir archivos?** Si prefieres usar `UTL_FILE` en lugar de `APEX_MAIL` para el CSV, necesito el nombre del directorio y permisos.